UNIVERSITY OF COPENHAGEN



Københavns Universitet

Fast and Efficient Boolean Unification for Hindley-Milner-Style Type and Effect Systems

Madsen, Magnus; van de Pol, Jaco; Henriksen, Troels

Published in:
Proceedings of the ACM on Programming Languages

DOI:
10.1145/3622816

Publication date:
2023

Document version
Publisher's PDF, also known as Version of record

Document license:
CC BY

Citation for published version (APA):
Madsen, M., van de Pol, J., & Henriksen, T. (2023). Fast and Efficient Boolean Unification for Hindley-Milner-Style Type and Effect Systems. *Proceedings of the ACM on Programming Languages*, *7*(OOPSLA2), Article 240. https://doi.org/10.1145/3622816

Download date: 01. Oct. 2026







Fast and Efficient Boolean Unification for Hindley-Milner-Style Type and Effect Systems

MAGNUS MADSEN, Aarhus University, Denmark
JACO VAN DE POL, Aarhus University, Denmark
TROELS HENRIKSEN, University of Copenhagen, Denmark

As type and effect systems become more expressive there is an increasing need for efficient type inference. We consider a polymorphic effect system based on Boolean formulas where inference requires Boolean unification. Since Boolean unification involves semantic equivalence, conventional syntax-driven unification is insufficient. At the same time, existing Boolean unification techniques are ill-suited for type inference.

We propose a hybrid algorithm for solving Boolean unification queries based on Boole's Successive Variable Elimination (SVE) algorithm. The proposed approach builds on several key observations regarding the Boolean unification queries encountered in practice, including: (i) most queries are simple, (ii) most queries involve a few flexible variables, (iii) queries are likely to repeat due similar programming patterns, and (iv) there is a long tail of complex queries. We exploit these observations to implement several strategies for formula minimization, including ones based on tabling and binary decision diagrams.

We implement the new hybrid approach in the Flix programming language. Experimental results show that by reducing the overhead of Boolean unification, the compilation throughput increases from 8,580 lines/sec to 15,917 lines/sec corresponding to a 1.8x speed-up. Further, the overhead on type and effect inference time is only 16% which corresponds to an overhead of less than 7% on total compilation time. We study the hybrid approach and demonstrate that each design choice improves performance.

CCS Concepts: • **Theory of computation** → **Program semantics**.

Additional Key Words and Phrases: Boolean unification, Hindley-Milner type systems, type inference

ACM Reference Format:
Magnus Madsen, Jaco van de Pol, and Troels Henriksen. 2023. Fast and Efficient Boolean Unification for Hindley-Milner-Style Type and Effect Systems. *Proc. ACM Program. Lang.* 7, OOPSLA2, Article 240 (October 2023), 28 pages. https://doi.org/10.1145/3622816

1 INTRODUCTION

"I feel a need… a need for speed." — *Lt. Pete "Maverick" Mitchell, Top Gun*

Programming languages with strong static type and effect systems help programmers to reason about their programs and to catch bugs at compile-time. As the reach of static type and effect systems expands, there is a need for fast and efficient type inference algorithms. Hindley-Milner style type and effect systems support parametric polymorphism while allowing principal type inference with Algorithm W [Damas 1984; Hindley 1969; Milner 1978]. Algorithm W is based on syntactic unification [Martelli and Montanari 1982] but can be extended to support unification over other equational theories [Odersky et al. 1999; Sulzmann 2001]. This paper explores how to

Authors' addresses: Magnus Madsen, magnusm@cs.au.dk, Aarhus University, Aarhus, Denmark; Jaco van de Pol, jaco@cs.au.dk, Aarhus University, Aarhus, Denmark; Troels Henriksen, athas@sigkill.dk, University of Copenhagen, Copenhagen, Denmark.



This work is licensed under a Creative Commons Attribution 4.0 International License.
© 2023 Copyright held by the owner/author(s).
2475-1421/2023/10-ART240
https://doi.org/10.1145/3622816

perform fast type and effect inference for the Flix programming language, a language which uses Boolean formulas to encode effects and uses *Boolean unification* to compute most-general unifiers inside Algorithm W [Madsen and van de Pol 2020]. The Flix effect system can express pure, impure, and effect polymorphic functions. For example, we can express that a function is *pure*:

```
def add(x: Int32, y: Int32): Int32 \ Pure = x + y
```

We can also express that a function is *impure* (i.e. has an unspecified IO effect):

```
def sayHello(name: String): Unit \ IO = println("Greetings: $name!")
```

We can express that a function has an effect in a region:

```
def push(x: a, l: MutList[a, r]): Unit \ r
```

The push function appends an element x to a mutable list l where the list belongs to some region r. The Flix type and effect system supports *effect polymorphism*, i.e. the effect(s) of a higher-order function may depend on the effect(s) of its function argument(s) [Lucassen and Gifford 1988]:

```
def map(f: a -> b \ ef, l: List[a]): List[b] \ ef
```

The effects of map depends on the effects of the function argument f. For example, if f is pure then the application of map is also pure. We can also express function composition:

```
def >>(f: a -> b \ ef1, g: b -> c \ ef2): a -> c \ (ef1 and ef2)
```

Note that the type and effect assigned to » is more precise than those of row-based effect systems and avoids the effect poisoning problem, where the inferred effects of f may impact the inferred effects of g and vice versa [Wansbrough and Jones 1999]. As these examples illustrate, Flix uses Boolean formulas to model effects. Individual effects, e.g. reading from or writing to a region, are represented as rigid variables, i.e. uninterpreted constants, in the Boolean formulas. During type and effect inference, Algorithm W has to compute the most-general unifier of two effects, i.e., of two Boolean formulas. Thus, Algorithm W has to solve *Boolean unification problems*.

The Boolean unification problem is the following: Given two Boolean formulas φ and ψ, compute the most-general unifier (MGU) of $\varphi \stackrel{?}{\equiv} \psi$, i.e. a substitution S such that $S(\varphi) \equiv_{\mathcal{B}} S(\psi)$.

A Boolean unification query, arising from the Flix Standard Library, could be:

$$x \wedge y \wedge z \stackrel{?}{\equiv} x \wedge w$$

We can see that $[x \mapsto \text{true}, y \mapsto \text{true}, z \mapsto \text{true}, w \mapsto \text{true}]$ is a solution. Indeed, so is $[x \mapsto \text{false}, y \mapsto \text{false}, z \mapsto \text{false}, w \mapsto \text{false}]$. Another solution is $[x \mapsto \text{false}]$ where y, z and w can be chosen freely. Yet, other solutions are $[y \mapsto w, z \mapsto w]$ and $[w \mapsto y \wedge z]$. But the *best* solution would be *a most general* solution, one from which all other solutions can be obtained. Fortunately, such an mgu exists: $S = [w \mapsto (w \wedge \neg x) \vee (x \wedge y \wedge z)]$. We can verify that this is a solution since:

$$\begin{aligned} S(x \wedge y \wedge z) &= S(x \wedge w) \\ x \wedge y \wedge z &= x \wedge ((w \wedge \neg x) \vee (x \wedge y \wedge z)) \\ x \wedge y \wedge z &= x \wedge y \wedge z \end{aligned}$$

The solution is most-general, which means that all other solutions are instances of it, i.e. can be obtained from it via substitution. In fact, a Boolean unification query may have multiple *equally* most-general solutions. This means that there could be many solutions, maybe infinitely many, but they are all instances of each other, i.e. Boolean unification is *unitary* [Martin and Nipkow 1989].

Boolean unification is a generalization of SAT. If we have formula φ, we can determine if it is satisfiable by solving the Boolean unification query $\varphi \stackrel{?}{\equiv} \text{true}$. If there is a unifier then φ is SAT, otherwise it is UNSAT. Hence, Boolean unification is NP-hard. While there is a large research community around SAT and SMT solvers and a plethora of industrial strength, off-the-shelf SAT

and SMT solvers available [Barbosa et al. 2022; de Moura and Bjørner 2008], we find no Boolean unification solvers. Fortunately, we can solve Boolean unification problems using two classic algorithms: Boole's Successive Variable Elimination (SVE) Algorithm [Martin and Nipkow 1989] or Löwenheims Method [Löwenheim 1908].

Using Boolean formulas in a type and effect system presents two practical challenges: (i) an expression may have multiple equivalent types (since it may contain Boolean formulas of which many are equivalent), and (ii) Boolean unification may lead to very large unifiers. The former is an HCI problem: it can be difficult for humans to spot when two syntactically distinct Boolean formulas are equivalent and hence when two types are equivalent. Unfortunately, such reasoning is often required while programming. The latter is a performance concern that may make type and effect inference impractically slow. In this paper, we set out to solve both challenges.

We empirically study the Boolean unification queries that arise during type and effect inference on the Flix Standard Library. Based on the study, we make several important observations, including: (i) most unification queries involve few variables, (ii) most unification queries repeat (i.e. the same unification query will occur multiple times during type and effect inference), and (iii) there is a long tail of complex queries that rarely repeat.

Although Boolean unification remains NP-hard, we exploit these observations to develop a collection of strategies that enable fast and efficient Boolean unification during type and effect inference and monomorphization in Flix. The key idea is to switch between two variants of the SVE algorithm. In cases where the formulas involve few variables, we use SVE equipped with a pre-computed table of minimal formulas to compute the unifier. In contrast, in cases where there are many variables, we switch to a variant of SVE that uses binary decision diagrams (BDDs) [Akers 1978] together with the Quine-McCluskey (QMC) algorithm [Quine 1952] to compute the unifier.

We implement these strategies in the Flix programming language compiler and experimentally evaluate their performance impact. The experimental results show that by reducing the overhead of Boolean unification, the compilation throughput increases from 8,580 lines/sec to 15,917 lines/sec corresponding to a 1.8x speed-up. We also show that the total overhead on type and effect inference time is only 16% which corresponds to an overhead of less than 7% on total compilation time.

In summary, the contributions of this paper are:

- (**Application**) We provide a gentle introduction to Boolean unification and its use in Hindley-Milner style type and effect systems.
- (**Empirical Study**) We perform an empirical study of the Boolean unification queries that arise during type and effect inference. We observe that: (a) the combination of Algorithm W and SVE quickly leads to exponential blow-up, (b) most unification queries have relatively few variables, and (c) with suitable use of minimization strategies the unifiers encountered during type and effect inference can be kept small.
- (**Data Set**) We provide a data set of real-world Boolean unification queries that arise from the Flix type and effect system when applied to the Flix Standard Library.
- (**Strategies**) We propose several strategies for improving the performance of type and effect inference based on Boolean unification. A key idea is a hybrid implementation of the SVE algorithm that uses tabling for queries with few variables and BDDs together with the QMC algorithm for queries with many variables.
- (**Implementation**) We implement the strategies in the Flix programming language and run several experiments on the Flix Standard Library.
- (**Evaluation**) We experimentally evaluate the performance impact of each strategy. The results suggest that a combination of strategies outperforms any single strategy.

2 MOTIVATION

We now illustrate how Boolean unification is used for type and effect inference in Flix.

2.1 Setting the Scene: Iterators in Flix

We will demonstrate the challenges presented by Boolean unification with an example based on Flix's Iterator module.[1] Our goal is to explain, in detail, how a complex Boolean unification query can arise during type and effect inference.

A Flix iterator is a stateful data structure that allows iteration through one or more underlying collections. Flix has iterators for Lists, Chains, Sets, Maps, and more. Iterators support a set of *non-terminal* and *terminal* operations. A non-terminal operation transforms an iterator. For example, we can map a function over an iterator to produce a new iterator, in which the function will be applied to every element of the original iterator. Non-terminal operations are *lazy*, i.e., no evaluation happens until the iterator is *forced*. A terminal operation aggregates the iterator into a value. For example, we can turn an iterator into a List, or combine the elements with a fold. In total, the Iterator module spans 779 lines of code and offers 42 functions.

Here is a small Flix program that uses the Iterator module:

```
def main(): Unit \ IO =
    let l = List#{"Apple", "Pear", "Mango"};
    let f = (i, s) ->  if (isEven(i)) String.toUpperCase(s) else s;
    let m = region rc {
        List.iterator(rc, l) |>
        Iterator.mapWithIndex(f) |>
        Iterator.toList
    };
    println(m)
```

The program works as follows: We bind l to a list literal which contains the three strings Apple, Pear, and Mango. Next, we define a local function f which takes two arguments: an integer i and a string s. If the integer i is even then it returns s in all uppercase. Otherwise, it returns s unchanged. Next, we introduce a region capability rc[2]. We then construct a new iterator on l in region rc. In Flix, all mutable data, including stateful iterators, must be associated with a region. We then map the function f over the iterator. This operation is non-terminal and happens lazily. Ultimately, we convert the iterator back to an immutable list, store the result in the local variable m, and print m. Thus, the program prints APPLE :: Pear :: MANGO :: Nil.

What's interesting is the types and effects of the functions that operate on the Iterator. In Flix, the type of Iterator is:

```
enum Iterator[t: Type, ef: Bool, r: Bool]
```

which has three type arguments: t which is the type of the elements of the iterator, ef which is the *latent effect* of the iterator, and r which is the region the iterator is associated with.

The signature of the List.iterator function is:

```
def iterator(rc: Region[r], l: List[a]): Iterator[a, r, r] \ r
```

[1] https://github.com/flix/flix/blob/master/main/src/library/Iterator.flix

[2] Flix has *region capabilities* (of kind Type) which are values and *regions* (of kind Bool) which are rigid Boolean variables. Each region capability is *indexed* by a region. In other words, the type of a region capability is of the form Region[r] for some rigid Boolean variable r. We say that reading or writing to memory in such a region has effect r.

The List.iterator function takes a region capability *rc*, associated with the region r, an immutable list l, and returns a new iterator with latent effect r and region r. Internally, the function allocates a reference cell in region r that points to the first Cons-cell of the list. Hence, calling the function has an effect in r, but the iterator also has latent effect r since to evaluate it we must read and update the internal reference cell.

The signature of the Iterator.mapWithIndex function is:

```
def mapWithIndex(f: (Int32, a) -> b \ ef2, iter: Iterator[a, ef1, r]):
    Iterator[b, ef1 and ef2, r] \ r
```

The List.mapWithIndex function takes a function f from elements of type a to type b with effect ef_2 and an iterator with elements of type a and latent effect ef_1. It returns a new iterator with elements of type b and with latent effect $ef_1 \wedge ef_2$. Notably, the List.mapWithIndex function is lazy, hence the ef_2 effect is accumulated in the iterator and does not occur until a terminal operation is applied to the iterator.

Finally, the signature of the Iterator.toList function is:

```
def toList(iter: Iterator[a, ef, r]): List[a] \ (ef and r)
```

The Iterator.toList function aggregates all elements of the iterator into a list and thereby *forces* evaluation of the latent effect. The signature of the function captures that it has two effects: the latent effect of the iterator ef and an effect in the region of the iterator r.

2.2 Type and Effect Inference with Boolean Unification

The definition of Iterator is:

```
enum Iterator[a: Type, ef: Bool, r: Bool] {
    case Iterator(Region[r], Unit -> Step[a] \ (r and ef))
}
```

The Iterator[a, ef, r] data type has three type parameters: a which is the type of the elements the iterator produces, ef which is the *latent* or *accumulated* effect of the iterator, and r which is the region associated with the iterator. The accumulated effect ef is a Boolean formula that captures the effects that are currently suspended in the iterator and will occur when a terminal operation is applied. The region can be thought of as a rigid, i.e. uninterpreted, Boolean constant.

The iterator value is a pair of two components: a region capability of type Region$[r]$ and a step function of type Unit $\to$ Step$[a] \setminus r \wedge ef$. Informally, the step function produces the next element in the iterator, if there is one. The function has effect $r \wedge ef$ which means that it has the latent effect of the iterator together with an effect in the region associated with the iterator. More precisely, the step function returns an element of the Step$[a]$ data type which is defined as:

```
enum Step[a] {
    case Ans(a)
    case Skip
    case Done
}
```

The Step has three variants: Ans(a), Skip, and Done. In reverse order, the step function returns Done when the iterator is exhausted and will not produce any more elements. Calling the step function again will continue to produce the Done value. The step function returns Skip when the

current element is to be skipped because of a filtering operation applied to the iterator previously. Calling the step function again may produce a different outcome for the next element. Finally, the step function returns an Ans(a) when there is an element a. The use of the Step data type makes it simple to implement many operations on iterators that would otherwise be complex.

With the definitions of Iterator[a, ef, r] and Step[a] in place, we present the implementation of the List.mapWithIndex function:

```
def mapWithIndex(f: (Int32, a) -> b \ ef2, iter: Iterator[a, ef1, r]):
    Iterator[b, ef1 and ef2, r] \ r =
    let Iterator(rc, iterF) = iter;
    let ix = ref 0 @ rc;
    let step = x -> match x {
        case Ans(a) => { let i = deref ix; ix := i + 1; Ans(f(i, a)) }
        case Skip   => Skip
        case Done   => Done
    };
    let iterF1 = () -> step(iterF());
    Iterator(rc, iterF1)
```

At a high-level, the mapWithIndex function returns a new iterator where the function f is (lazily) applied to the elements of the iterator together with their respective index (i.e. position). Importantly, as the example shows, if the iterator has accumulated effect ef_1 and the function f has latent effect ef_2 then accumulated effect of the returned iterator is $ef_1 \wedge ef_2$[3].

The mapWithIndex has to keep track of the index of the next element. To do so, it allocates a counter, i.e., a reference cell ix in region rc that points to an Int32. Initially the counter is zero. The mapWithIndex defines a local step function. The Skip and Done cases are trivial. In the Ans case, when there is an element, it retrieves the current value of the counter and stores it in i, it then increments the counter, and finally returns a new Ans element with the result of applying f to the current index i and the current element a.

Importantly, the inferred type and effect of the step function is:

$$\text{step} : \text{Step}[a] \to \text{Step}[b] \setminus r \wedge ef_2$$

Since step has an effect in r, due to the read/write of ix, and because it calls f which has effect ef_2. We use step to define a new function iterF1 which will pull an element of the original iterator and apply step to it. The type and effect of this function is:

$$\text{iterF}_1 : \text{Unit} \to \text{Step}[b] \setminus r \wedge ef_1 \wedge ef_2$$

Since its effect is the combination of iterF, which has effect $r \wedge ef_1$, and step which has effect $r \wedge ef_2$.

In the last line of List.mapWithIndex we pass the region capability rc and $iterF_1$ to the Iterator constructor. Recall from the definition of Iterator that this constructor has the type scheme:

$$\text{Iterator} : \forall r, ef.\ \ \text{Region}[r] \to (\text{Unit} \to \text{Step}[a] \setminus r \wedge ef) \to \text{Iterator}[a, ef, r] \qquad (\star)$$

To determine whether the application of Iterator to rc and iterF_1 is typeable, we must instantiate the type scheme and determine if we can unify the two function types:

$$\text{Unit} \to \text{Step}[b] \setminus r \wedge ef_1 \wedge ef_2 \stackrel{?}{\equiv} \text{Unit} \to \text{Step}[a] \setminus r \wedge ef$$

[3]The declared type of mapWithIndex is slightly less general than it could be, as we shall see shortly.

The two functions types are unifiable if the following two Boolean formulas are (cf. Section 1):

$$r \wedge ef_1 \wedge ef_2 \stackrel{?}{\equiv} r \wedge ef$$

Why does this unification problem occur? The reason is that when we declared Iterator we specified that it has three type parameters: t, ef, and r. We also specified that the effect of the step function should be $ef \wedge r$. Now, inside List.mapWithIndex, we have a concrete step function with inferred effect $r \wedge ef_1 \wedge ef_2$. Can we match these two formulas? Luckily, as it turns out, there is a most-general unifier:

$$S : [ef \mapsto (ef \wedge \neg r) \vee (ef_1 \wedge ef_2 \wedge r)]$$

If we apply S to the type scheme from ($\star$), we conclude that the type of the returned Iterator is:

$$\mathsf{Iterator}[b, (ef \wedge \neg r) \vee (ef_1 \wedge ef_2 \wedge r), r]$$

This type might look strange at first glance[4], but let us see what happens when we unfold the definition of Iterator. Remember that the signature of Iterator is $\mathsf{Iterator}[a, ef, r]$ where the step function has effect $ef \wedge r$. Thus, the unfolded type of the step function is:

$$\mathsf{Unit} \to \mathsf{Step}[a] \setminus (r \wedge ((ef \wedge \neg r) \vee (ef_1 \wedge ef_2 \wedge r)))$$

which simplifies to:

$$\mathsf{Unit} \to \mathsf{Step}[a] \setminus (ef_1 \wedge ef_2 \wedge r)$$

hence the inferred type is exactly what we would expect!

We have seen how a Flix program can give rise to a non-trivial Boolean unification problem. Next we discuss that solving such queries can lead to scalability issues; even for relatively simple queries like the above.

2.3 How bad can it get? Boolean Unifiers Explode

In the example, we encountered the Boolean unification problem:

$$r \wedge ef_1 \wedge ef_2 \stackrel{?}{\equiv} r \wedge ef$$

and we reported that a most-general unifier was: $[ef \mapsto (ef \wedge \neg r) \vee (ef_1 \wedge ef_2 \wedge \mathrm{r})]$. This is the unifier computed by Flix *after* applying all the minimization strategies presented in this paper. If we compute the unifier with the *original* version of Flix, i.e., with the SVE algorithm with only very ad-hoc rewrite rules, then we obtain the equally most-general unifier:

$$\begin{aligned}
ef \quad \mapsto \quad & (r \wedge ef_1 \wedge \neg(r \wedge ef_2 \wedge ((r \wedge ef_2 \wedge \neg r) \vee (\neg(r \wedge ef_2) \wedge r))) \wedge ef_2) \\
& \vee (ef \wedge \neg(((r \wedge ef_1 \wedge \neg((r \wedge ef_2) \wedge ((r \wedge ef_2 \wedge \neg r) \vee (\neg(r \wedge ef_2) \wedge r)))) \wedge ef_2 \wedge \neg r) \\
& \vee (\neg(r \wedge ef_1 \wedge \neg(r \wedge ef_2 \wedge ((r \wedge ef_2 \wedge \neg r) \vee (\neg(r \wedge ef_2) \wedge r))) \wedge ef_2) \wedge r))) \\
ef_1 \quad \mapsto \quad & ef_1 \wedge \neg(r \wedge ef_2 \wedge ((r \wedge ef_2 \wedge \neg r) \vee (\neg(r \wedge ef_2) \wedge r)))
\end{aligned}$$

Consequently, the inferred type of mapWithIndex becomes: `Iterator[b, ef, r]` where ef is the above Boolean formula which has 47 connectives! What can and does happen is that such substitutions snowball out of control. Specifically, when Algorithm W progresses, it continually applies and composes such substitutions leading to exponentially larger and larger formulas. Dramatically, every call to SVE, for every flexible variable, leads to an exponentially doubling in formula size!

[4]What is the intuition for this seemingly strange type? Where does the $ef \wedge \neg r$ part come from? The intuition is the following: We can think of false, which is the least element of the Boolean algebra, as representing "all regions". Hence, if a function has the effect "false" it has an effect in all regions. The unifier captures that *if* r = false *then* ef can be chosen freely: it can be pure or have an effect in one or multiple regions because the "false" effect already accounts for all possible effects.

2.4 Worst-case Programs

We build intuition for how complex Boolean unification queries can arise from type and effect inference on Flix programs. In particular, we consider two "worst-case" inputs: one with many flexible variables and one with many rigid variables.

An effect with multiple flexible variables occurs when we have higher-order functions:

```
let f = x -> a -> b -> c -> a(b(c(x))); // effect polymorphic in a, b, c.
let g = y -> u -> v -> w -> u(v(w(y))); // effect polymorphic in u, v, w.
let h = if (true) f else g
```

Here the effects of f and g contain three flexible variables each, one for each of the function arguments, a, b, c, and u, v, w. The if-then-else forces Boolean unification of two formulas that together have six unique variables (3 + 3 = 6). As we have seen several times already, such unification problems lead to large unifiers if computed naïvely. Note that it is important to have a proper unification problem. For example, calling f with three pure functions poses no challenge for Boolean unification, since each effect variable is simply instantiated to true.

An effect with multiple rigid variables occurs when we have nested regions:

```
region rc1 { region rc2 { region rc3 {
    def f(x) = { Array.new(rc1, 1); Array.new(rc2, 1); Array.new(rc3, 1) };
    def g(x) = { Array.new(rc3, 1); Array.new(rc2, 1); Array.new(rc1, 1) };
    let h = if (true) f else g
}}}
```

Here the effects of f and g are equivalent, but they are not *syntactically* the same, i.e., one formula is $r_1 \wedge r_2 \wedge r_3$ whereas the other formula is $r_3 \wedge r_2 \wedge r_1$. The if-then-else forces Boolean unification of two formulas which together contain three unique variables. Since there are no flexible variables this reduces to a SAT problem.

We can combine the two ideas to produce worst-case inputs with many flexible and rigid variables.

2.5 Why do we need Most-General Unifiers?

We require most-general unifiers because we want to ensure that the type and effect system has principal types, i.e. every typeable expression has most-general type. If we were to choose a less general type during type and effect inference, then some programs – which are valid according to the declarative type system – would no longer type check. For example, for List.mapWithIndex we could compute and use the substitution $[ef_1 \mapsto \text{true}, ef_2 \mapsto \text{true}, ef \mapsto \text{true}]$ or $[ef_1 \mapsto ef, ef_2 \mapsto ef]$ (cf. Section 1) but these would impose unnecessary restrictions on the typeable programs. That is why we must compute most-general unifiers.

We can now present the main challenge of the paper:

> **Research Question:** How can we implement fast and efficient Boolean unification for use in type and effect systems based on Hindley-Milner and Algorithm W?

2.6 Why model effects with Booleans? – The "Effect Poisoning" Problem

Flix uses Boolean formulas to model effects to avoid the "effect poisoning" problem [Wansbrough and Jones 1999] inherent in many other systems, including those based on row polymorphism [Hillerström and Lindley 2016; Leijen 2014]. In a nutshell, the effect poisoning problem is when the effect

of one expression pollutes the effect typing of another, unrelated expression. For example, in the expression: `if (e1) e2 else e3`, if e_2 has the Print effect, a row-based effect system may (soundly, but imprecisely) infer that e_1 and e_2 also have the Print effect. Thus, the effect of e_2 "poisons" the effects of e_1 and e_3. In contrast, the Flix type and effect system, is *fine-grained* and *compositional*, since effects are constructed from sub-expressions [Madsen and van de Pol 2020]. For example, Flix is able to type e_1 and e_3 as pure, regardless of what effects e_2 has.

The upshot is that more expressions are typed as pure. This has two practical benefits: (i) it means that more expressions are eligible for inlining, and (ii) it enables a program construct called *purity reflection* which can be used for automatic and selective lazy and/or parallel evaluation [Madsen and van de Pol 2023].

As mentioned, the Flix type and effect system is based on Boolean algebras, which have most-general unifiers, but one might wonder whether another algebraic structure could have been used. Unfortunately, potential alternative structures, e.g. Heyting algebras [Ghilardi 1999], commutative idempotent monoids [Baader and Büttner 1988], and distributive lattices [Gerhard and Petrich 1994], are not unitary (i.e. lack most-general unifiers).

3 BACKGROUND

3.1 The Boolean Unification Problem

Boolean Formulas. Let $\mathcal{B} = \{\text{true}, \text{false}\}$ and let *BoolVar* denote a countable set of Boolean variables ($\beta, \ldots$). The set *Formula* of Boolean formulas ($\varphi, \psi, \ldots$) is defined inductively by:

$$\varphi, \psi \in \mathit{Formula} = \text{true} \mid \text{false} \mid \beta \mid \neg\varphi \mid \varphi \wedge \varphi \mid \varphi \vee \varphi$$

We write $\text{ftv}(\varphi)$ for the free variables in φ. A *valuation* ν for a formula φ is an assignment of Booleans $\mathcal{B}$ to all variables in $\text{ftv}(\varphi)$. In this way, a Boolean formula of n variables denotes a Boolean function $\mathcal{B}^n \to \mathcal{B}$. Two Boolean formulas φ_1 and φ_2 are *equivalent* (written $\varphi_1 \equiv_{\mathcal{B}} \varphi_2$) if they describe the same function. That is, if $\forall \nu.\, \nu(\varphi_1) = \nu(\varphi_2)$ where ν must be a valuation of both φ_1 and φ_2. For example, we have:

$$\text{true} \equiv_{\mathcal{B}} x \vee \neg x \qquad \neg(x \vee y) \equiv_{\mathcal{B}} \neg x \wedge \neg y$$

Substitutions. A *substitution* S is a partial mapping $\mathit{BoolVar} \to \mathit{Formula}$. We write $\{x_i \mapsto \psi_i \mid i \in I\}$ for the substitution S, such that $S(x_i) = \psi_i$. We extend substitutions to a total function on formulas, and write $S(\varphi)$ for the result of applying S to φ. The composition $S_2 \circ S_1$ of substitutions S_1 and S_2 is defined such that $(S_2 \circ S_1)(\varphi) = S_2(S_1(\varphi))$. We will often drop $\circ$ and () and simply write $S_2 S_1 \varphi$. We write $S_1 \equiv_{\mathcal{B}} S_2$ (substitution S_1 and S_2 are equivalent) iff for all $x \in \beta$, $S_1(x) \equiv_{\mathcal{B}} S_2(x)$. We write $S_1 \leq_{\mathcal{B}} S_2$ (substitution S_1 is more general than S_2) iff there exists a substitution S_3 such that $S_3 S_1 \equiv_{\mathcal{B}} S_2$.

Definition 3.1 (Boolean Unification Problem). Given two Boolean formulas φ and ψ, the *Boolean Unification Problem* $\varphi \stackrel{?}{\equiv} \psi$ is to compute a unifier, i.e. a substitution S such that $S(\varphi) \equiv_{\mathcal{B}} S(\psi)$ or to report that no such substitution exists. Similarly, a solution S for a set of Boolean Unification Problems $\{\varphi_i \stackrel{?}{\equiv} \psi_i \mid i \in I\}$ is a single substitution S with $S(\varphi_i) \equiv_{\mathcal{B}} S(\psi_i)$ (for all $i \in I$).

A solution S is a most general unifier (mgu), if $S \leq_{\mathcal{B}} S'$ for all other unifiers S'.

Note that the most general unifier is not unique, not even modulo $\equiv_{\mathcal{B}}$. This means that two most general unifiers S_1 and S_2 are not necessarily equivalent, i.e. we do not necessarily have $S_1 \equiv_{\mathcal{B}} S_2$. We only have that $S_1 \leq_{\mathcal{B}} S_2$ and $S_2 \leq_{\mathcal{B}} S_1$, i.e. we can transform one into the other (and vice versa) via another substitution.

Table 1. Examples of Boolean Unification

		Most-General Unifier of $\varphi \stackrel{?}{\equiv} \psi$	
φ	ψ	Minimal MGU	Naïvely Computed by SVE
true	false	*no unifier*	*no unifier*
x	$\neg x$	*no unifier*	*no unifier*
$x \wedge y$	true	$\{x \mapsto \text{true}, y \mapsto \text{true}\}$	*same*
$x \wedge y$	false	$\{x \mapsto x \wedge \neg y\}$	*same*
$x \vee y$	true	$\{x \mapsto \neg y \vee x\}$	*same*
$x \vee y$	false	$\{x \mapsto \text{false}, y \mapsto \text{false}\}$	*same*
$x \vee y$	$x \wedge y$	$\{x \mapsto y\}$	$\{\lvert x\rvert = 3\}$
$x \wedge y$	z	$\{z \mapsto x \wedge y\}$	$\{\lvert x\rvert = 18, \lvert y\rvert = 5\}$
$x \wedge y$	$z \wedge w$	$\{w \mapsto (w \wedge \neg z) \vee (x \wedge (y \wedge z)), x \mapsto x \wedge (\neg y \vee z)\}$	$\{\lvert w\rvert = 35, \lvert y\rvert = 8\}$
$x \wedge y$	$z \vee w$	$\{w \mapsto (w \vee \neg z) \wedge (z \vee (x \wedge y)), x \mapsto x \vee z, y \mapsto y \vee z\}$	$\{\lvert w\rvert = 52, \lvert x\rvert = 13, \lvert y\rvert = 1\}$
$x \wedge y \wedge z$	$u \wedge v$	$\{\lvert u\rvert = 8, \lvert v\rvert = 3\}$	$\{\lvert u\rvert = 105, \lvert v\rvert = 35, \lvert x\rvert = 8\}$

Theorem 3.2 (Martin and Nipkow [1989]). *Boolean Unification is decidable. Moreover, if $\varphi \stackrel{?}{\equiv} \psi$ is solvable then there exists a most general unifier.*

This result goes back to Boole himself [Boole 1847]. Boole's algorithm is known as the *Successive Variable Elimination* (SVE) algorithm.

We have already seen several examples of Boolean unification problems and their most-general unifiers. A few more examples are shown in Table 1. For all cases, it is easy to verify that S is a *unifier*, i.e. $S(\varphi) \equiv_{\mathcal{B}} S(\psi)$. It is less obvious to check that the result is a *most-general* unifier. Table 1 shows how quickly unifiers can grow as the formulas increase in size.

3.2 Boolean Unification is not Syntactic Unification

Unification is an important building block in type inference algorithms for the Hindley-Milner (HM) type system, and their implementation is well studied, for example in the form of Algorithms J and W [Milner 1978]. In traditional HM, unification is *syntactic*—we try to find substitutions that make two terms structurally identical, and data structures such as *union-find* allows this to be done efficiently. In contrast, Boolean unification is *non-syntactic*—two terms can look very different, yet express the same Boolean function. This means we cannot use the syntactic structure to guide unification, which makes the problem significantly more complicated, and means we cannot use existing optimized techniques for handling unification of types in the HM type system.

We give two examples to illustrate that Boolean unification is different from syntactic unification:

Example 3.3 (No Occurs Check). Given the Boolean unification problem:

$$x \wedge x \stackrel{?}{\equiv} x$$

A syntactic unification procedure would fail with the "*occurs check*" since the variable x occurs recursively inside the $x \wedge x$ formula. However, such a recursive occurrence poses no problem for Boolean unification, since semantically $x \wedge x = x$ and $x \stackrel{?}{\equiv} x$ has the most-general unifier $[x \mapsto x]$.

We remark that just because there is no occurs check does not mean the Boolean unification is not sensitive to the occurrence of unification variables inside formulas. For example, one might think that the Boolean unification problem $x \stackrel{?}{\equiv} \varphi$ has the solution $[x \mapsto \varphi]$, but this can be false if x occurs within φ. For example, the unification problem $x \stackrel{?}{\equiv} \neg x$ does *not* have any unifiers.

```
def sve(f: Formula, flexvs: List[Var]): Substitution = match flexvs {
    case Nil => // Base case: Determine if the residual is unsatisfiable.
        if (!satisfiable(f)) Substitution.empty
        else throw BooleanUnificationException("No Unifier")
    case x :: xs => // Variable elimination case: solve for x.
        let f0 = Substitution.singleton(x, mkFalse())(f)
        let f1 = Substitution.singleton(x, mkTrue())(f)
        let se = sve(mkAnd(f0, f1), xs)
        let fx = mkOr(se(f0), mkAnd(mkVar(x), mkNot(se(f1))))
        let sx = Substitution.singleton(x, minimize(fx))
        sx ++ se
}
```

Fig. 1. The Successive Variable Elimination Algorithm

We might get — and we did — the bright idea that we can speed up Boolean unification in the following way: Initially try syntactic unification, which is significantly cheaper than full Boolean unification. If it succeeds, we use its computed substitution as the most-general unifier. Otherwise, try a proper Boolean unification algorithm.

Regrettably this idea does not work, as the following example illustrates:

Example 3.4 (Syntactic Unifiers are not Most-General). Given the Boolean unification problem:

$$x \wedge y = y \wedge x$$

a syntactic unifier is $[x \mapsto y, y \mapsto y]$. However, this is *not* the most-general unifier. A most-general unifier is the identity, i.e. $[x \mapsto x, y \mapsto y]$.

In summary, we have seen:

- The non-existence of a *syntactic* unifier does not imply that there is not a *semantic* unifier.
- A *syntactic* unifier is not necessarily the most-general unifier of two Boolean formulas.

3.3 The Successive Variable Elimination (SVE) Algorithm

We now present the Successive Variable Elimination (SVE) algorithm. The key insight in the algorithm is that Boolean unification can be reduced to the following matching problem [Martin and Nipkow 1989]:

> The unification problem $\varphi \stackrel{?}{\equiv} \psi$ has the same solutions as $(\varphi \oplus \psi) \stackrel{?}{\equiv} \text{false}$.

where $\oplus$ is the Xor-operator.

The following theorem shows how we can eliminate a single variable from a matching problem. We use the traditional notation $\varphi[x := \psi]$ for the substitution application $\{x \mapsto \psi\}(\varphi)$.

Theorem 3.5 ([Martin and Nipkow 1989, Equation 2.12]). *$\varphi \stackrel{?}{\equiv} false$ has an mgu if and only if $\varphi[x := false] \wedge \varphi[x := true] \stackrel{?}{\equiv} false$ has an mgu S. In that case, the mgu of $\varphi \stackrel{?}{\equiv} false$ is the composition of substitutions, $S \circ \{x \mapsto (\varphi[x := false] \vee (x \wedge \neg\varphi[x := true]))\}$.*

Fig 1 shows the SVE algorithm. If the Boolean unification problem is $\varphi \stackrel{?}{\equiv} \psi$ then the initial call to the algorithm is: $\text{sve}(\varphi \oplus \psi, \text{ftv}(\varphi) \cup \text{ftv}(\psi))$. Note that the order in which variables are eliminated, i.e. the order of $\text{ftv}(\varphi) \cup \text{ftv}(\psi)$ when converted to a list, impacts the computed most-general unifier.

The algorithm recursively applies Theorem 3.5, eliminating all variables from f one by one. Given input formula f, we compute two modified copies f_0 and f_1 and recursively call SVE with $f_0 \wedge f_1$ (line 8). Note that this effectively doubles the size of the argument to SVE in each call, leading to an exponential blow-up. Even worse, if the unification succeeds (i.e., doesn't throw an exception), these large formulas appear in the result substitution fx for x, constructed at line 9, 10. To avoid this scenario, and to make the algorithm practical, Boolean formulas must be aggressively simplified during SVE. Hence, on line 10 when we extend the computed substitution (i.e. the mgu) we call the minimize function on the formula.

Minimization of formulas is difficult, and depends heavily on the formula representation. Therefore, the algorithm in Fig 1 has been generalized in two ways: (i) it takes an explicit list of variables to eliminate (flexvs), a point we return to in Sec 3.4, and (ii) it is parameterized by the mkTrue, mkFalse, mkNot, mkAnd, mkOr, and minimize functions. The latter allows us to write the SVE algorithm once and enables us to experiment with different representations of Boolean formulas.

3.4 Flexible and Rigid Variables

We typically solve a slightly generalized Boolean unification problem: Given two Boolean formulas φ and ψ, we want to compute their most-general unifier $\varphi \stackrel{?}{\equiv} \psi$ where some variables in φ and ψ are *flexible*, i.e. can be instantiated, and some variables are *rigid*, i.e. cannot be instantiated.

We can think of rigid variables as *uninterpreted constants* that are only equal to themselves. Flix uses rigid variables to model regions and user-defined effects. Every region, as introduced by the `region` construct, is associated with a unique rigid variable. Regions can of course nest. Similarly, every user-defined effect is associated with a unique rigid variable. Happily, Boolean unification with rigid variables remains unitary, i.e. such problems also have most-general unifiers. If all variables in a Boolean unification problem are rigid then the problem reduces to a SAT problem (for "rigid" φ, $\varphi \stackrel{?}{\equiv} \mathsf{false}$ iff φ is not satisfiable). Thus, the ratio of flexible to rigid variables represents a continuum from Boolean unification to SAT. As we shall see, most queries have many flexible variables and few rigid variables, hence our application is conceptually "closer" to Boolean unification than to SAT.

Example 3.6. The unification problem $x! \stackrel{?}{\equiv} \mathsf{true}$ where $x!$ denotes a rigid variable has no unifier.

Example 3.7. The unification problem $x \wedge y! \stackrel{?}{\equiv} \mathsf{true}$ where x is a flexible variable and $y!$ is a rigid variable has no unifier.

Example 3.8. The unification problem $x \stackrel{?}{\equiv} y \vee z$ where x, y, and z are flexible variables has the most-general unifier $[x \mapsto y \vee z]$. If x is made rigid then the unifier is no longer valid. However, there is another unifier: $[y \mapsto (x! \wedge (y \vee \neg z)), z \mapsto x! \wedge z]$.

The SVE algorithm, shown in Fig 1, is already adapted to solve Boolean unification queries with rigid variables. We simply pass it the set of flexible variables to be eliminated.

3.5 Essential Variables

A key feature of the Hindley-Milner type system is its support for parametric polymorphism based on let-generalization [Damas 1984; Hindley 1969; Milner 1978].

The (T-Let) rule from [Madsen and van de Pol 2020] is:

$$\frac{\Gamma \vdash e_1 : \tau_1 \mid \varphi_1 \quad \Gamma, x : \mathsf{gen}(\Gamma, \tau_1) \vdash e_2 : \tau_2 \mid \varphi_2}{\Gamma \vdash \mathsf{let}\, x = e_1 \,\mathsf{in}\, e_2 : \tau_2 \mid \varphi_1 \wedge \varphi_2} \quad \text{(T-Let)}$$

where $\mathsf{gen}(\Gamma, \tau) = \forall\alpha_1, \cdots, \forall\alpha_n.\forall\beta_1, \cdots, \forall\beta_n.\tau$ with $\{\alpha_1, \cdots, \alpha_n, \beta_1, \cdots, \beta_n\} = \mathsf{ftv}(\tau) \setminus \mathsf{ftv}(\Gamma)$.

The rule states that if we can type e_1 with the monotype τ_1 under some context Γ then we can *generalize* τ_1 to a type scheme $\text{gen}(\Gamma, \tau_1)$ and use it as the type of x when type checking e_2. Importantly, the definition of gen specifies that we can only generalize type variables in τ_1 which are not bound by Γ. In the standard Hindley-Milner type system this check is purely syntactic: we simply compute the free variables of τ_1 and of Γ. However, in the presence of Boolean formulas, we can no longer rely on syntactic occurrences, since a variable could syntactically occur, but be semantically irrelevant. Instead, we need to work with essential variables:

Definition 3.9 (Essential Variables). Given a Boolean formula f and a variable x that syntactically occurs within f, we say that x is *essential* if there is a valuation ν of f where $\nu[x \mapsto \text{true}]$ and $\nu[x \mapsto \text{false}]$ distinguishes the value of f, i.e. $(\nu[x \mapsto \text{true}])(f) \neq (\nu[x \mapsto \text{false}])(f)$.

Example 3.10. Given the formula $f = x \wedge (y \vee \neg y)$ the variable x is essential whereas y is non-essential. An equivalent formula is $f = x$ in which y does not occur.

During type and effect inference with Algorithm W and Boolean unification, a Boolean formula could arise in which a variable occurs syntactically, but is not essential. This could "unfairly" block let-generalization, leading to incompleteness of type and effect inference.

We can solve this problem in one of two ways:

- (**Lazy**) We compute the essential variables of the context Γ at every let-generalization step, instead of merely computing its syntactically free variables.
- (**Eager**) We choose a representation of Γ which ensures that we can readily determine the essential variables: (A) we can ensure that the formulas in Γ are minimal and hence their syntactically free variables coincide with essential variables, or (B) we can use a representation of formulas, e.g. BDDs, which makes it trivial to identify the essential variables.

In this paper, we use the eager strategy based on a combination of (A) and (B). Notably the ad-hoc rewrite rules used in [Madsen and van de Pol 2020] are not guaranteed to produce a context Γ where all syntactically occurring variables are also essential variables. Thus, in theory, their implementation is incomplete.

3.6 The Boolean Unification Workbench: An Online Tool

We have found that it can be difficult to build intuition about Boolean unification. To help ourselves and the community, we have developed an online tool, called the Boolean Unification WorkBench (BUWB), which allows for experimentation with Boolean unification queries.

The tool is available at:

https://flix.dev/buwb/

A screenshot is shown on the right.

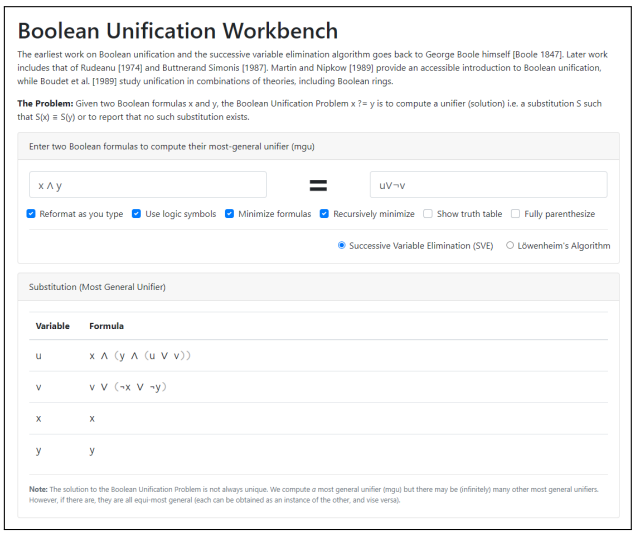


Fig. 2. Boolean Unification WorkBench (BUWB)

4 EMPIRICAL STUDY

4.1 Strategies

This section describes a progression of strategies for implementing Boolean unification in Flix. The progression largely follows its actual development over time, and is intended to show how we needed both a theoretically well-founded approach and a variety of heuristics to obtain good performance. We measure *throughput* in lines of code per second of the *entire* compiler, not just the parts that involve Boolean unification[5]. We consider seven different strategies, with corresponding compiler throughput shown in Fig 3. We ran the Flix compiler with OpenJDK 19 on a Ryzen 4750U CPU. We perform ten runs per strategy and take their arithmetic mean.

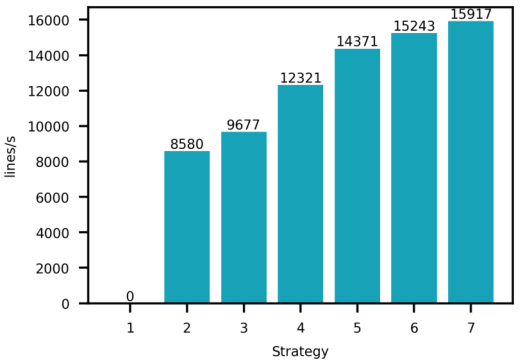


Fig. 3. Compiler throughput using each of the seven implementation strategies for Boolean unification. For Strategy 2, Boolean unification takes up ~47% of the total runtime, while for Strategy 7 it is only ~7% .

Strategy 1 corresponds to the original approach used by Flix [Madsen and van de Pol 2020], and uses Boolean unification with SVE and ad-hoc algebraic simplification of intermediate formulas. However, the effects used in the Flix standard library have grown much more complicated since the original publication, and the formulas produced by SVE are now large, and ad-hoc simplification proves incapable at reducing their size sufficiently. To type-check the current Flix Standard Library, this strategy now requires more than 512GiB of memory—exceeding what we have available in our largest machine—and must therefore be considered wholly inadequate.

Strategy 2 addresses the problem of formula size by using a precomputed table containing the smallest formula representation of every binary function of four variables or fewer. Such a table comprises 2^{2^4} elements and takes up 4MiB of space. We find the minimal representation of a formula by evaluating it for each of the $2^4 = 16$ possible assignments of variables to values, and then treating this as a 16-bit index into the table. As formulas involving more than four variables are quite rare, as discussed shortly, this strategy is effective, and results in a reasonable running time.

Strategy 3 augments strategy 2 by adding two components: (i) a *query cache* that stores the computed solutions to unification queries and simply reproduces the solution when the query is encountered again, and (ii) specialized handling of certain very common unification queries, such as unifying syntactically identical types or formulas consisting of single variables.

Strategy 4 discards the use of caching and special cases, and instead attacks the problem of formula size by using *Binary Decision Diagrams* (BDDs) as the intermediate representation of Boolean formulas during SVE. A BDD is a concise representation of a Boolean function, which is canonical modulo variable ordering. However, translation of BDDs back to formulas can be costly[6].

Strategy 5 improves the translation of BDDs into Boolean formulas by using the Quine-McCluskey (QMC) algorithm [McCluskey 1956]. QMC uses prime implicants to compute a formula that, while not guaranteed to be minimal, often produces very small formulas.

Strategy 6 re-introduces the query cache and special cases of Strategy 3.

[5]Boolean unification is used not just in the type checker, but also during monomorphization.

[6]We have to translate BDDs to Boolean formulas (or more precisely types) since it is impractical to work with BDDs throughout a production compiler.

Strategy 7 brings back the use of the table from Strategy 2. Specifically, for a query involving up to four variables we use SVE with table-based minimization whereas if the query has more than four variables we switch to a BDD-based SVE implementation.

While the improvements from Strategies 5–7 are individually minor, their speedup contribution, using Strategy 2 as baseline, is almost 1.3×. The total speedup from Strategy 2 to 7 is over 1.8×.

4.2 Quantifying the Problem

To explain the effectiveness of the strategies above, we conduct an empirical study of the Boolean unification queries that arise during type and effect inference in the Flix compiler. Specifically, we collect data to answer the following questions:

- **Q1**: What is the distribution of variables in the queries?
  - How many variables are *flexible*?
  - How many variables are *rigid*?
- **Q2**: How often do queries repeat?
  - How many queries are *syntactically* identical?
  - How many queries are *semantically* identical?
- **Q3**: What is the structure of the queries (if any)?
  - What types of queries occur most frequently?

We used this information to develop the strategies discussed in the previous section. An important goal is that we should try to answer the above questions *independent of the specific Boolean unification algorithm* such that the answers are reusable no matter the specific implementation strategy we choose. We achieve this for every question other than Q2(a).

Our decision to benchmark the entire type checker, rather than unification in isolation, requires justification. If we benchmarked unification in isolation, the results could be misleading: the unification problems we solve are not available all at once, but arrive incrementally, and give rise to new unification problems, due to Algorithm W. In particular, the shape and size of the next query depends on the answer to previous queries. This means that the latency of solving small problems matters, and concurrency is only intermittently possible. These properties are hard to model in a synthetic benchmark targeting just unification. So it is best to stick to the overall goal: making the entire compiler faster as experienced by the end-user.

Now let us begin with an in-depth discussion of our empirical test subject.

4.3 The Flix Standard Library

We use the Flix Standard Library[7] as the subject for the empirical evaluation. We describe the structure of the library in detail to illustrate that it is a non-trivial code base suitable for study.

The Flix Standard Library consists of several modules. Each module may contain data type declarations, type class declarations, instance declarations, and functions. Data type declarations and class declarations are usually short, hence the bulk of the library consists of functions.

The library includes an assortment of immutable and mutable collections. Immutable collections include Lists, Sets, and Maps, but also Chains, Vectors, and non-empty lists (Nel) and non-empty chains (Nec)[8].

[7] https://api.flix.dev/ and https://github.com/flix/flix/tree/master/main/src/library

[8] A List is an immutable singly-linked list of elements. A list offers fast access to the head element and fast consing. Each node has a link to its successor. A Chain is immutable tree of elements. A chain offers fast concatenation, but slow access to the head element. Each node has a link to its predecessor and successor. A Vector is an immutable sequence of elements (i.e. an immutable array) which offers fast lookup of any element, but consing and concatenation is slow.

Mutable collections include MutList, MutDeque, MutSet, and MutMap. As already discussed, the library also has a rich Iterator module. All mutable data is associated with a region and functions that operate on mutable data are typically region polymorphic. Higher-order functions that operate on immutable and mutable data are almost always effect polymorphic.

Additionally, the Flix Standard Library contains three "sub-libraries": (i) an implementation of channels (Channel.flix) used for concurrency and parallelism, (ii) an implementation of a command-line arguments parser (GetOpt.flix), and (iii) a complete Datalog engine (everything in Fixpoint). We briefly expand on the latter. An important feature of the Flix programming language is its support for first-class Datalog constraints enriched with lattice semantics [Madsen and Lhoták 2020]. To this end, the Flix Standard Library includes a sophisticated Datalog engine. The implementation consists of a JIT compiler that translates first-class Datalog program values into Relational Algebra Machine (RAM) programs. The RAM IR is a C-like language extended with relational algebra operators. After translation to RAM, the JIT compiler does index selection and performs a few optimization passes on the RAM IR. Finally, the RAM program is executed by an interpreter.

In summary, Flix takes a "batteries-included" approach to its Standard Library, and while the language is still relatively young, it already includes a lot of non-trivial functionality.

Table 2 shows an overview of the modules in the Flix Standard Library. For brevity, we have only included modules with more than 125 lines of code. The table has the following columns: The Module column gives the name of the module (i.e. filename). The Lines column shows the total number of lines of Flix source code in the module. The Functions column shows the total number of functions in the module. The count includes named public and private functions, but not lambda expressions. The count also includes functions with default implementations in type class declarations and functions implemented in type class instances. The Pure, Impure, and Effect Polymorphic columns show the number of pure functions, impure functions, and effect polymorphic functions in the module, respectively. Together the three columns add up to the total number of functions in the module, since a function is either pure, impure, or effect polymorphic. As an example, the row for the List.flix module shows that it consists of 1,406 lines of source code corresponding to 151 functions, of which 85 are pure, 1 is impure, and 65 are effect polymorphic.

In total, the Flix Standard Library is 35,629 lines of code. The library contains 3,267 functions of which 1,911 (58.5%) are pure, 132 are impure (4%), and 1,224 (37.5%) are effect polymorphic.

A few things in Table 2 may stand out. We briefly address them here: (i) A few modules, including Array, Chain, List, MutDeque, MutList, and Vector, have *one* impure function. What is this single impure function? It is shuffle which must be impure due to its non-deterministic behavior.

(ii) The modules for mutable data structures, e.g. MutList, MutSet, and MutMap, have few impure functions. How so, if they use mutation? This is because, except for shuffle, they are all effect polymorphic (region polymorphic, to be specific).

We believe that the Flix Standard Library is one of the largest libraries for a programming language with a type and effect system. For comparison, we measured the size of the standard libraries in Frank [Convent et al. 2020], Eff [Pretnar 2015], Links [Hillerström and Lindley 2016; Lindley and Cheney 2012], Effekt [Brachthäuser et al. 2018, 2020], and Koka [Leijen 2014, 2017]. Table 3 shows the result.

The first three columns are self-explanatory. The Library column shows the number of lines of source code in the standard library for that language[9], the ★-lang column shows the total number of lines of source code *written in that language.*[10]. In other words, this count includes not only the

[9]We measured the line count by running `find -name '*.ext'|xargs cat|wc -l` in the directory which we identified as the standard library. As `ext`, we used `.fk` for Frank, `.eff` for Eff, `.effekt` for Effekt, `.kk` for Koka, `.links` for Links, and `.flix` for Flix.

[10]We measured the line count in the same way, but on the whole repository

Table 2. The Flix Standard Library.

		Overview of Functions in the Flix Standard Library			
Module	**Lines**	**Functions**	**Pure**	**Impure**	**Effect Polymorphic**
Applicative.flix	185	14	8	0	6
Array.flix	1,677	135	3	1	131
Benchmark.flix	199	17	8	8	1
BigDecimal.flix	226	21	21	0	0
BigInt.flix	359	30	30	0	0
Boxable.flix	238	30	30	0	0
Chain.flix	959	111	58	1	52
Char.flix	132	16	16	0	0
Choice.flix	150	12	12	0	0
Concurrent/Channel.flix	490	17	2	15	0
Console.flix	596	54	48	6	0
Debug.flix	128	3	2	1	0
DelayList.flix	1,367	110	54	1	55
DelayMap.flix	930	82	38	0	44
Environment.flix	146	16	16	0	0
Eq.flix	237	38	38	0	0
File.flix	1,221	50	4	45	1
Fixpoint/Compiler.flix	366	9	6	0	3
Fixpoint/IndexSelection.flix	221	6	6	0	0
Fixpoint/Interpreter.flix	211	8	0	0	8
Fixpoint/Simplifier.flix	156	4	4	0	0
Fixpoint/Solver.flix	775	42	38	1	3
Fixpoint/VarsToIndices.flix	126	4	4	0	0
Float32.flix	347	34	34	0	0
Float64.flix	368	35	35	0	0
Foldable.flix	385	44	25	0	19
GetOpt.flix	421	26	24	0	2
Graph.flix	562	29	29	0	0
Group.flix	165	29	29	0	0
Hash.flix	182	23	23	0	0
Identity.flix	133	24	14	0	10
Int16.flix	402	45	45	0	0
Int32.flix	440	47	47	0	0
Int64.flix	456	48	48	0	0
Int8.flix	371	43	43	0	0
Iterator.flix	779	52	12	0	40
JoinLattice.flix	212	14	14	0	0
List.flix	1,406	151	85	1	65
Map.flix	988	118	50	0	68
MeetLattice.flix	213	14	14	0	0
MultiMap.flix	580	71	32	0	39
MutDeque.flix	474	41	5	1	35
MutList.flix	966	76	1	1	74
MutMap.flix	661	76	0	0	76
MutSet.flix	445	51	0	0	51
Nec.flix	1,031	129	63	1	65
Nel.flix	702	107	52	1	54
Option.flix	586	81	35	0	46
Order.flix	648	71	71	0	0
PartialOrder.flix	196	14	14	0	0
Prelude.flix	168	18	12	2	4
RedBlackTree.flix	1,076	84	44	0	40
Reducible.flix	290	38	20	0	18
Result.flix	452	52	20	0	32
SemiGroup.flix	169	25	25	0	0
Set.flix	642	80	41	0	39
String.flix	1,224	105	84	0	21
StringBuilder.flix	133	15	0	0	15
System.flix	141	11	0	11	0
ToString.flix	205	27	27	0	0
Validation.flix	294	35	17	0	18
Vector.flix	1,505	155	84	1	70
...	...	...	...	...	...
Totals	**35,629**	**3,267**	**1,911**	**132**	**1,224**

Table 3. Comparison of Standard Libraries in Programming Languages with Effect Systems.

		GitHub	*Lines of Source Code*	
Language	**Stars**	**Contributors**	**Library**	**⋆.lang**
Frank	240	6	—	4,827
Eff	787	14	—	8,826
Links	294	30	1,655	30,479
Effekt	174	12	1,837	6,186
Koka	2,500	25	15,371	105,741
Flix	1,700	52	35,629	128,566

standard library, but also the example programs and test cases found in the repository. This gives us an upper bound on the amount of source code written in that language – in its main repository.

The table suggests that Flix and Koka are the two largest programming language developments with type and effect systems, both in terms of their standard libraries and the overall amount of code written in them. The point we want to make is that the Flix Standard Library *is* a significant code base that contains real-world code which we can use as the basis for our experiments.

In addition to the Flix Standard Library, we also include approximately 30,000 lines of unit tests. We include these unit tests because we want to exercise the entire compiler pipeline, including the Monomorphization phase which also relies on Boolean unification.

With these details in place, we can now move on to the research questions. To collect data, we instrument the Flix compiler to record all Boolean unification queries for later analysis. The recorded queries are available in the artifact.

4.4 Q1: What is the distribution of variables in the queries?

We count the number of variables in each recorded query. Fig 4 shows the results[11]. It is immediately clear that the vast majority of queries (over 95%) involve at most two variables. Intuitively, the number of variables is proportional to the number of effect-polymorphic function arguments, as well as the static nesting depth of regions. Most functions are first order and pure, which results in many trivial queries. Further, most higher-order functions take only a single functional argument. Finally, deeply nested uses of regions are rare. The fact that the vast majority of queries involve fewer than five variables explains why moving from Strategy 1 to Strategy 2 (i.e. using tabling) suddenly makes inference practical again for the extended Flix Standard Library.

We see that only 0.2% of the queries involve more than four variables — yet this is the cause of the speed-up when moving from Strategy 3 to Strategy 4 (cf. Section 4.1). This shows that the long tail of large queries is a major cost center, and their efficient handling is crucial for performance.

The histogram in Fig 5 shows that most of the queries exclusively involve flexible variables. The figure shows that, regardless of the number of variables in the query, typically more than 90% have only flexible variables. If a query only has flexible variables then all the work is done in SVE and the residual SAT problem is trivial.

[11]The precise queries encountered during type and effect inference can vary depending on the chosen strategy, as multiple representations of the same Boolean function can exist. However, all strategies except Strategy 2 produce a unique formula for each function, and Strategy 2 produces unique formulas up to four variables, so the impact of any differences is minor. To answer this question, we recorded the queries encountered by the Flix compiler while using Strategy 7.

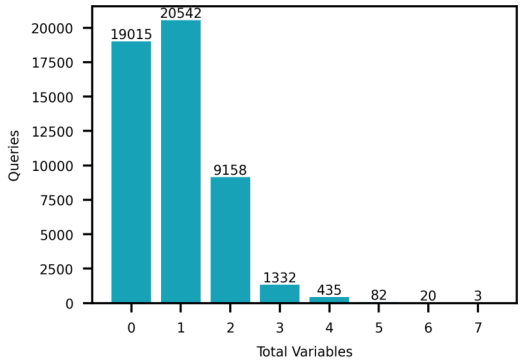


Fig. 4. Variable distribution in unification queries. The X-axis shows the number of variables. The Y-axis shows the number of queries.

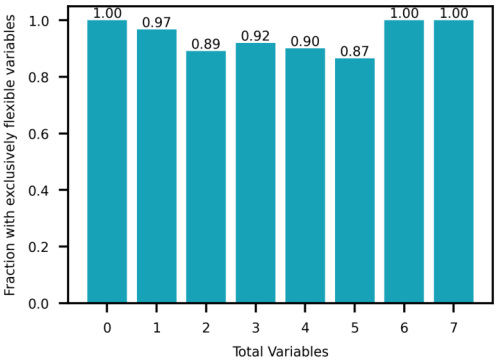


Fig. 5. Fraction of queries with exclusively flexible variables. The X-axis is the number of variables. The Y-axis is the ratio.

4.5 Q2: How often do unification queries reoccur?

We find that out of 50587 recorded Boolean unification queries there are only 380 *syntactically* unique unification queries. Of these, there are only 89 *semantically* unique queries.

This suggests that caching should be very effective, but that is not supported by the results in Section 4.1. Specifically, Strategy 2 only provides a minor speed-up over Strategy 3, and similarly Strategy 6 only provides a minor speed-up over Strategy 5.

To explain this phenomenon, we must look at the structure of the unification queries.

4.6 Q3: What is the structure of the unification queries?

Given that many queries repeat, we may wonder if their distribution is uniform: i.e. are any two formulas equally likely? The answer turns out to be no, as shown in the table on the right. While the table shows that *many simple queries repeat*, we also find that there is a *long tail of complex queries*. In particular, we find that there are 326 syntactically unique queries that repeat ten times or less. Recall that this is 326 out of 380 total syntactically unique queries.

Query	Rigid Vars	Hits
$\text{true} \stackrel{?}{\equiv} \text{true}$	$\emptyset$	17,855
$x_0 \stackrel{?}{\equiv} \text{true}$	$\emptyset$	9,538
$x_0 \stackrel{?}{\equiv} x_0$	$\{x_0\}$	5,485
$x_0 \stackrel{?}{\equiv} x_1$	$\{x_1\}$	5,424
$x_0 \stackrel{?}{\equiv} x_0$	$\emptyset$	1,560
375 more rows		

This explains why caching is not as effective as might be hoped for: as illustrated in Fig 6, there are a few repeat queries for high-variable-count queries. And these are the most complex to solve. Specifically, Fig 6 can be understood as the "cache miss rate". With fewer than four variables, the miss rate is low (i.e. many queries repeat). With four or more variables, the miss rate increases from 7% to 22%, 45%, and 100%.

So high-variable-count queries are rarely cached, but are they expensive to compute? We use the cactus plot in Fig 7 to show that (unfortunately) they are the most expensive queries. The plot shows how many queries (individually) can be solved within a given time budget. For example, with Strategy 7, about 40,000 queries can each be solved in less than 10^3 ns = 1 us, i.e. there are 40,000 queries where it takes less than 1 microsecond to solve each. The plot shows while most queries — perhaps 99% can be solved in less than 1 millisecond — there is a significant tail of complex queries that are very expensive to solve.

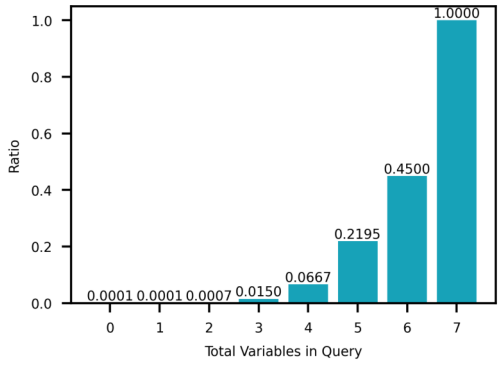


Fig. 6. Ratio of semantically unique queries for each size of query. The X-axis shows the number of variables in each query, and the Y-axis shows, for each query with that number of variables, the number of semantically unique queries divided by the total number of queries.

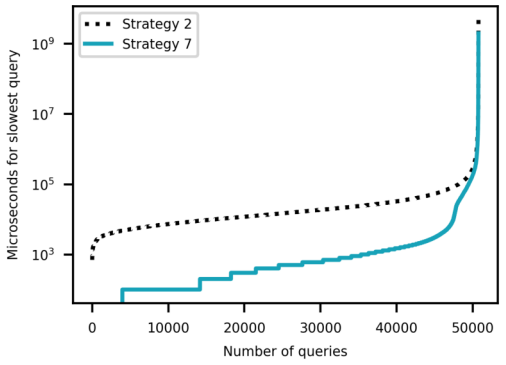


Fig. 7. The number of queries that can be solved, with a specific strategy, within a given time budget. The X-axis shows the number of solved queries. The Y-axis shows the time in nanoseconds.

This explains why our hybrid approach is valuable: The use of query caching, hard-coded special cases, and tabling significantly reduces the cost of simple queries. Although each saving might be small, as we have seen, the vast majority of queries fall into this category. On the other hand, the use of BDDs and QMC brings a much greater overhead, but is the only effective solution at dealing with the few, but complex queries in the tail. (As a side-comment, we believe that the "jaggedness" in the plot for Strategy 7 is due to the use of the hard-coded special cases– which are extremely fast for the specific queries they cover.)

Inspecting the structure of the queries, we find that the query true $\stackrel{?}{\equiv}$ true occurs 18701 times, but the query false $\stackrel{?}{\equiv}$ false only occurs 314 times! This is explained by the fact that pure code is associated with true whereas impure code is associated with false, and there is significantly more pure than impure code in the Flix Standard Library. We also find that 57% of the queries are of the form $\varphi \stackrel{?}{\equiv} \psi$ where φ and ψ are true, false, or a variable. These, and similar observations, are what we use to implement the hard-coded special cases. Specifically, the special cases are queries that are extremely common, can be handled without any expensive computation, and where their checks are determined by careful profiling.

4.7 Summary

In summary, we make the following observations about the Boolean unification queries:

- (**Observation I**) Most queries typically involve fewer than three flexible variables.
- (**Observation II**) Most queries reoccur: out of 50587 queries, we find 380 syntactically unique queries and 89 semantically unique queries.
- (**Observation III**) Most *common* queries are simple: out of 50587 queries more than 57% of the queries are of the form $\varphi \stackrel{?}{\equiv} \psi$ where φ and ψ is true, false, or a variable.
- (**Observation IV**) While most queries are common there is a long tail of complex queries.

5 IMPLEMENTATION TECHNIQUES

5.1 Putting Everything Together (Strategy 7)

We now provide a technical overview of the best strategy, Strategy 7. The overall algorithm is shown in Figure 8. The input is given as two Boolean formulas φ and ψ represented as types in the applicative style[12]. We call the type-based representation the "syntactic query". The input also consists of a set of variables that should be treated as rigid.

Given a syntactic query (i.e. ψ and φ), we first consider nine hard-coded special cases. These include queries like true $\stackrel{?}{\equiv}$ true, but also $x \stackrel{?}{\equiv} y$ where either x or y is flexible. If one of these special cases match, we have the solution, and we are immediately done. Next, we determine the number of variables in the query (i.e. in φ and ψ). We ignore whether they are flexible or rigid, and whether they are essential or non-essential. We know in the common case they are flexible and essential. If there are four or fewer variables, we do a syntactic lookup in the syntactic cache. If there is a hit, we are done. Otherwise, we run the SVE algorithm with tabling. Note that the syntactic cache can contain multiple entries for the same Boolean function. Nevertheless, in our experiment the syntactic cache ends up containing only 380 entries. Caches are per compiler run (i.e. they are not persisted to disk). If, on the other hand, there are more than four variables then we convert the Boolean formula to a BDD. We do a lookup in the BDD-based cache. If there is a hit, we are done. Note that since BDDs have a unique representation, the BDD cache is minimal (and for our experiment ends up containing 89 entries after type checking).

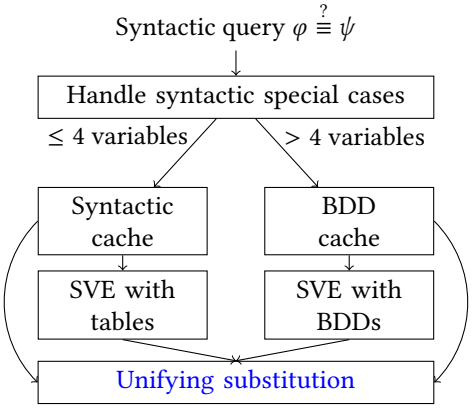


Fig. 8. Overview of the Algorithm, showing only paths that result in successful unification.

If there is no hit, we run SVE with the BDD-based representation. Once complete, we translate the resulting substitution back into a type-based (i.e. syntactic) representation using the Quine-McCluskey algorithm.

To measure the cost of Boolean unification, we compare Strategy 7 versus a *no-op* implementation where every query succeeds with the empty substitution. In total, we find that Strategy 7 imposes a 16% overhead on type and effect inference time. This corresponds to an overhead of less than 7% on total compilation time. In contrast, [Madsen and van de Pol 2020] report 40% and 10% overhead for the original Flix implementation. Hence, Strategy 7 outperforms [Madsen and van de Pol 2020] by a significant margin on a significantly expanded Flix Standard Library (from 10,000 to 35,000 lines, cf. Section 6).

BDD Libraries. We experimented with three BDD libraries: BeeDeeDee [Lovato et al. 2014], JavaBDD [Whaley 2023], and PJBDD [Beyer et al. 2021]. We could not get BeeDeeDee to work– it would always run out of memory. We got JavaBDD to work, but its throughput was less than 50 lines per second. Finally, we found PJBDD, which is newer than the two other libraries by a decade, and worked well enough.

In the next two sections, we explain the relevant implementation details for Boolean formula minimization with precomputed tables and with Binary Decision Diagrams and Quine-McCluskey.

[12]In Flix, effects (i.e. Boolean formulas) are types of kind Bool and they are represented using type constructors. For example, $ef_1 \wedge ef_2$ is represented as the type constructor And applied to ef_1 which is then applied to ef_2.

5.2 Pre-Computed Tables of Minimal Formulas (Strategy 2 + 7)

A Boolean formula with n variables has 2^n different valuations. By mapping these to either true or false, one obtains all 2^{2^n} possible Boolean functions on n inputs. For small n, it should be feasible to make a table and store for each Boolean function the *smallest* Boolean formula that represents this formula[13]. Assume we have such a table. We can use it to minimize a Boolean formula φ with at most n variables as follows: (1) Evaluate φ on all 2^n possible valuations, creating a "function profile" consisting of 2^n bits. (2) Lookup the smallest formula for this "function profile" in the table. This provides a minimal and canonical representation for φ. Note that for a formula containing $n = 3, 4$ or 5 different variables, step (1) requires 8, 16 or 32 evaluations, respectively. Each evaluation is quite simple (it requires at most one full pass per evaluation), so this step is still feasible. The corresponding lookup tables would roughly have 256, 64k and 4G entries. This back-of-the-envelope calculation shows that it should be completely feasible to equip a compiler with a table of minimal formulas for 4 variables, but the table for 5 variables would be largish.

We now specify what connectives we consider, and how we define the minimal size. We restrict ourselves to formulas built with $\neg$, $\wedge$, and $\vee$ in so-called Negation Normal Form. That is, negations are pushed inside. In particular, the smallest formulas are variables x or their negations $\neg x$. Larger formulas are built by binary $\wedge$ and $\vee$. The size of the formula is simply defined as the number of occurrences of $\wedge$ and $\vee$ (we don't count negations).

This design choice has two advantages. First, we believe that formulas in this fragment are relatively easy to read for the Flix programmer, since they are short, negations only happen in the leaves, and conjunction/disjunction are monotonic. Second, this choice enables the generation of all minimal formulas by a dynamic programming technique: A minimal formula of size m must be either a conjunction or a disjunction of two minimal formulas of size p and q, where $m = p + q + 1$. Now we simply generate all minimal formulas by increasing size, and store them in a global table. The $2n + 2$ formulas of size 0 are true, false, n variables, and their n negations. To generate minimal formulas of size $m > 0$, we simply try all combinations of minimal formulas φ of size p and ψ of size q (such that $m = p + q + 1$), and check if $\varphi \wedge \psi$ and $\varphi \vee \psi$ are already in the table. If so, we could already construct them by a smaller formula. If not, we found a minimal representation for them, and store it in the table. In order to minimize startup-time for the compiler, we pre-compute the minimal formula table for 4 variables once and for all, and make it part of the compiler code. We implemented the dynamic programming technique in a dedicated C-program, which could generate all 65 536 minimal formulas of 4 variables within 3.5 seconds. In the table on the right, we display how many formulas are generated for a certain size. The process terminates after formulas of size 15, since we have a minimal formula for all 65 536 formulas. For size 15, we needed to try (among others) all combinations of 7x7 formulas, leading to $11768^2 = 138\,485\,824$ combinations, which constitutes the worst pair (p, q). With the table in hand, we can implement the minimize function used in SVE.

size	number
0	10
1	48
2	256
3	940
4	2048
5	5248
6	8672
7	11768
8	10592
9	11536
10	5472
11	6304
12	960
13	1472
14	96
15	114

5.3 Binary Decision Diagrams and Conversion to Formulas (Strategy 5–7)

Binary Decision Diagrams (BDDs) represent Boolean functions by directed acyclic graphs [Akers 1978]. The leaves of the DAG can be 0 or 1 (representing false and true). Internal nodes B of the DAG are labeled with proposition variables x, and point to two children B_1 and B_2, representing the subfunctions when x is true or false, respectively. Each node can be thought of as an if-then-else

[13]What is "smallest" depends on the permitted connectives and a definition of size.

$\mathsf{ITE}(x, B_1, B_2)$. BDDs can be *reduced* by avoiding useless tests $\mathsf{ITE}(x, B, B)$, and enforcing maximal sharing between subterms. Moreover, BDDs can be *ordered* by ensuring that on all paths starting from the root, the tests appear in a fixed total order $x_1, x_2, \ldots, x_n$. Gaps are allowed, i.e. a variable can be skipped, but duplicate tests along a path are not allowed.

The main advantage of BDDs comes from the theoretical result that every Boolean function has a unique reduced ordered BDD (assuming a fixed variable ordering). Hence, logical equivalence of BDDs can be tested by a simple structural check on the canonical representation. This makes satisfiability and tautology tests on BDDs trivial. Also, a Boolean variable x occurs *essentially* in some formula, iff x appears syntactically in a test in its BDD. Due to maximal sharing, BDDs often stay small in practice, although this may depend on an appropriate variable ordering. Note that for our use case, the number of variables is quite low. Logical operations (and, xor, not, etc.) can be performed directly on BDDs. Minimization is trivial (the identity operation). The most difficult operation for us is substitution: $B_1[x := B_2]$ can be computed on BDDs by computing a ternary operation $\mathsf{ITE}(B_2, B_1|_{x=1}, B_1|_{x=0})$. The subfunctions $B_1|_{x=v}$ can be computed in linear time, but the ITE-operation takes cubic time in the worst case.

Since the entire compiler is based on types which may contain Boolean formulas, we need an interface between formula representations and BDDs. A formula is translated to a BDD by a simple bottom-up process, applying logical operations on previously built BDDs for subformulas. The other way around is harder: given a BDD, one might build a formula in DNF (disjunctive normal form) as the disjunction over all paths in the BDD that lead to the 1-leaf. Each such path leads to a conjunction of (positive or negative) literals, corresponding to the tests on the path.

Note that the DNF representation of a formula is not unique. Even worse, the naïve DNF transformation described above would blow up, basically undoing all advantages of the sharing in the BDD representation. As an example, let us start with a formula $x \vee y$. Assuming $x < y$ in the ordering, the BDD will be $\mathsf{ITE}(x, 1, \mathsf{ITE}(y, 1, 0))$. Taking the disjunction over all paths to 1, we obtain the DNF $x \vee (\neg x \wedge y)$, which is correct, but larger than necessary.

Instead, we have implemented a variant of the Quine-McCluskey (QMC) algorithm [Quine 1952] to compute a small DNF for a BDD (representing the same Boolean function). This operates on literals and cubes. A literal is a Boolean variable (x) or its negation ($\neg y$). A cube is the set of valuations corresponding to a conjunction of literals (e.g. $x \wedge \neg y$). Note that a small conjunction corresponds to a large cube. The goal is to cover the original formula or BDD with a small set of large cubes (i.e. disjunction of conjunctions of literals).

Basically, the algorithm considers all conjunctions in increasing size for inclusion in the DNF. A cube can be included in the DNF if it is a prime implicant, i.e., a minimal conjunction that implies the Boolean function. The process terminates when the Boolean function is completely covered. In the example above, we would first consider the single-variable cubes x and y. Both imply $x \vee y$ and together cover it, so we would never consider the larger 2-variable cube $\neg x \wedge y$. The resulting DNF $x \vee y$ will be used in the Flix compiler and can be shown to the programmer.

While the translation to and from BDDs may sound expensive, it is not in the context of type and effect inference in Flix: First, the output to SVE tends to stay small, thanks to the use of BDDs + QMC. Second, the input stays small as well, either since it comes directly from the type and effect constraints (which tend to be small, at least for the programs we have seen), or it comes from the output of previous calls to SVE (which was kept small). Finally, during the SVE computation itself there is a potential blow-up, but it is mitigated by performing the whole computation on BDDs.

6 RELATED WORK

6.1 Boolean Unification

Early work on Boolean unification and the Successive Variable Elimination algorithm goes back to George Boole [Boole 1847]. Later work includes that of Rudeanu [1974] and Buttner and Simonis [1987]. Löwenheim presents an alternative algorithm for Boolean unification [Löwenheim 1908]. Löwenheim's algorithm computes a particular solution which is then generalized to a most-general unifier. In general, Löwenheim's algorithm leads to more complex unifiers than the SVE algorithm [Martin and Nipkow 1989]. An introduction to Boolean unification is provided by Martin and Nipkow [1989]. Baader [1998] studies the the computational complexity of Boolean unification. Additional background is provided by Boudet et al. [1989]; Robinson and Voronkov [2001].

While Boolean technology has been widely studied, including research into BDDs [Akers 1978; Bryant 1992], SAT [Biere et al. 2021], and SMT solvers [Moura and Bjørner 2011], Boolean unification has seen relatively little investment. Early applications of Boolean unification include circuit design and logic programming [Bockmayr 1991; Buttner and Simonis 1987; Simonis and Dincbas 1987].

Macii et al. [1998] presents, we believe, the first real-world study on the performance of Boolean unification. The paper compares the performance of Boole's algorithm, i.e. the SVE algorithm, versus Löwenheim's algorithm. Both algorithms are implemented based on BDDs and their performance is compared on 13 benchmarks. The benchmarks are described as circuit design problems. Like in our work, the benchmarks (i.e. queries) include both flexible and rigid variables which are referred to as "unknowns" and "parameters". In the benchmarks, the number of flexible variables range from 3 to 34 whereas the number of rigid variables range from 1 to 67. The largest query has 34 flexible variables and 67 rigid variables, but most queries are smaller. While their benchmarks are more complex than ours, by up to an order of magnitude, ultimately their data set only has 13 queries whereas ours has 50587 queries of which 380 are syntactically unique, and 89 are semantically unique. Notably, the most-general unifiers computed in their work are small. This supports our observation that most real-world Boolean unification problems have small unifiers: the challenge is just to compute them. Ultimately, Macii et al. [1998] conclude that "one can observe that the two methods are basically equivalent for large benchmarks. The greater the number of unknowns (i.e. flexible variables), the smaller the differences between the two methods. For small benchmarks, Boole's algorithm (i.e. SVE) seems to require less memory than Löwenheim's."

6.2 Type Systems and Type Inference

The Hindley–Milner type system was described by Hindley [1969] and Milner [1978]. Later, Damas [1984] studied the formal foundations of the system. The Hindley–Milner type system, with its support for type inference, has been widely successful in practice and is used as the formal foundation for several real-world functional programming languages, including Haskell, OCaml, and Standard ML. Over the years, many extensions have been proposed, including type classes [Wadler and Blott 1989], qualified-types [Jones 1994], and region-based memory management [Tofte and Talpin 1997].

We believe that Boolean unification was first used in a type system by de Vries et al., who used it to model uniqueness [de Vries et al. 2008]. In their system, Boolean formulas express uniqueness attributes where true means a variable is unique and false means a variable is non-unique, and a formula describes under what conditions a variable is unique. The type system is implemented in the Morrow compiler, but the paper does not address any performance aspects.

In this paper, we build on the work of Madsen and van de Pol who present a polymorphic type and effect system that uses Boolean formulas to represent effects and Boolean unification to support type inference [Madsen and van de Pol 2020]. In that paper, the authors rely on an ad-hoc collection

of simplification rules. We have shown that such ad-hoc rewrite rules do not scale. Specifically, when that paper was published, the Flix Standard Library consisted of 10,000 lines of code and had only a single effect: Impure. Today, the Flix Standard Library is 35,000 lines of code, and has multiple effects in the form of user-defined region capabilities. In this paper, we have presented a detailed analysis of the Boolean unification queries that arise during type and effect inference, and we have used this information to implement a hybrid approach to Boolean unification that scales beyond that of [Madsen and van de Pol 2020].

Madsen and van de Pol also propose a type system with support for relational nullable types [Madsen and van de Pol 2021]. While a nullable type system tracks whether an expression may evaluate null to based on its type, a *relational* nullable type system tracks whether an expression may evaluate to null based on its type and the type of other related expressions. The type system uses two Boolean formulas to capture when an expression may evaluate to null and when it may evaluate to a non-null. The system relies on Boolean unification to solve constraints over these formulas. The paper does not address any performance aspects.

We see several emerging applications of Boolean unification in type systems, and we believe our paper is the first to address the question of how to quickly and efficiently compute small most-general unifiers for use in type inference. We hope these results will be useful for others building type systems based on Boolean unification.

6.3 Effect Systems

There is a rich research literature on type and effect systems [Brady 2013; Convent et al. 2020; Hillerström and Lindley 2016; Kammar and Pretnar 2017; Kiselyov and Sivaramakrishnan 2018; Leijen 2014, 2017; Lindley et al. 2017; Lucassen and Gifford 1988; Plotkin and Pretnar 2009; Pretnar 2014; Rytz et al. 2012; Tofte and Talpin 1997]. We cannot hope to cover all aspects, instead we outline the major research directions. A major line of research has focused on *effect polymorphism*, which is the idea that the effects of a higher-order function depend on the effects of its function arguments [Brachthäuser et al. 2020; Lucassen and Gifford 1988; Rytz et al. 2012]. Another line of research has focused on *region-based memory management*, where the idea is to use effects to statically track memory, thus avoiding the need for garbage collection [Nielson et al. 1999; Tofte and Talpin 1997]. A recent line of research, which has received a lot of attention, is *algebraic effects* where the idea is that programmers should be able to define and interpret their own effects [Brachthäuser et al. 2020; Brady 2013; Hillerström and Lindley 2016; Leijen 2017; Plotkin and Pretnar 2009].

6.4 Programming Languages with Effect Systems

We have already discussed several programming languages with effect systems (cf. Section 4), including Frank [Convent et al. 2020], Eff [Pretnar 2015], Effekt [Brachthäuser et al. 2018, 2020], Koka [Leijen 2014, 2017], Links [Hillerström and Lindley 2016; Lindley and Cheney 2012], and Flix [Madsen and Lhoták 2020; Madsen and van de Pol 2020]. The languages range from small prototypes to large-scale developments with hundreds of thousands of lines of code. In terms of scale, Flix and Koka are most prominent. The Koka type and effect system is based on row-polymorphism [Gaster and Jones 1996], specifically using a form of scoped labels [Leijen 2005, 2017]. In our experience, row unification [Gaster and Jones 1996], unlike Boolean unification, is fairly efficient unless the rows become very long.

7 LESSONS LEARNED AND CONCLUSION

This paper has presented our work on a type and effect system based on Boolean unification, with a particular focus on its efficient implementation. We have distilled our experiences into the following lessons, in the hope that they will prove useful to future implementers of similar systems.

- (**Lesson 1**) The Flix type and effect system presented in Madsen and van de Pol [2020] relies on ad-hoc simplification rules for Boolean formulas. In their work, the authors report a 1.4x slowdown caused by Boolean unification on a 10,000 lines Flix Standard Library with only a single effect: impurity. Since then, the library has expanded to 35,000 lines of code and with user-defined regions. As shown by Strategy 1 that naïve approach to Boolean unification is no longer tractable.
- (**Lesson 2**) Binary Decision Diagrams (BDDs) offer an efficient representation of Boolean formulas which can be used during computation of Boolean unifiers with SVE. While switching to a purely BDD based approach makes type and effect inference for the Flix Standard Library tractable, it also imposes a significant overhead, much larger than the original of 1.4x reported in [Madsen and van de Pol 2020].
- (**Lesson 3**) BDD libraries tend to be optimized for large formulas with many variables, while we need to handle many relatively small formulas. Our experience with PJBDD [Beyer et al. 2021], JavaBDD [Whaley 2023], and BeeDeeDee [Lovato et al. 2014] showed that PJBDD offered the best performance for our use case.
- (**Lesson 4**) We exploited domain-specific insights to speed up computation of most-general unifiers in our setting. In particular, (i) most SVE queries are simple, (ii) most SVE queries involve few variables, and (iii) queries are likely to repeat due similar programming patterns.
- (**Lesson 5**) We improved performance of Boolean unification by a hybrid approach to SVE: When the number of variables is few, we use an explicit representation of Boolean formulas along with precomputed tables to quickly compute small unifiers. When more variables are involved, we fall back to a BDD based approach. Furthermore, we improved performance by caching the result of previous unification queries. Finally, for extremely common queries, it is worth hard-coding their solutions directly, before hitting the caches or invoking SVE.

In summary, our approach reduces the overhead of Boolean unification to 16% of type and effect inference time which corresponds to an overhead of less than 7% of total compilation time. Furthermore, we have shown that the hybrid technique beats any single technique on its own and that going from Strategy 2 to Strategy 7 increases compilation throughput from 8,580 lines/sec to 15,917 lines/sec corresponding to a 1.8x speed-up. Lastly, but most importantly, our approach remains tractable in the presence of a significantly expanded Flix Standard Library.

We believe that Boolean formulas are a fundamental structure with many potential applications in type (and effect) systems. In particular, Boolean formulas support most-general unifiers which can be computed by Boolean unification, thus making them a perfect match for Hindley-Milner style type systems. Hence, we hope that our results will be useful for future uses of Boolean unification.

DATA-AVAILABILITY STATEMENT

The software used to conduct the study in Section 4 is available on Zenodo [Madsen et al. 2023].

REFERENCES

Akers. 1978. Binary Decision Diagrams. *IEEE Trans. Comput.* C-27, 6 (jun 1978), 509–516. https://doi.org/10.1109/tc.1978.1675141

Franz Baader. 1998. On the complexity of Boolean unification. *Inform. Process. Lett.* 67, 4 (aug 1998), 215–220. https://doi.org/10.1016/s0020-0190(98)00106-9

Franz Baader and Wolfram Büttner. 1988. Unification in commutative idempotent monoids. *Theoretical Computer Science* 56, 3 (mar 1988), 345–353. https://doi.org/10.1016/0304-3975(88)90140-5

Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A Versatile and Industrial-Strength SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer International Publishing, 415–442. https://doi.org/10.1007/978-3-030-99524-9_24

Dirk Beyer, Karlheinz Friedberger, and Stephan Holzner. 2021. PJBDD: A BDD Library for Java and Multi-Threading. In *Automated Technology for Verification and Analysis*. Springer, 144–149. https://doi.org/10.1007/978-3-030-88885-5_10

Armin Biere, Marijn Heule, Hans van Maaren, and Toby Walsh (Eds.). 2021. *Handbook of Satisfiability - Second Edition*. Frontiers in Artificial Intelligence and Applications, Vol. 336. IOS Press. https://doi.org/10.3233/FAIA336

Alexander Bockmayr. 1991. Logic Programming with Pseudo-Boolean Constraints. (1991).

George Boole. 1847. *The mathematical analysis of logic*.

Alexandre Boudet, Jean-Pierre Jouannaud, and Manfred Schmidt-Schauss. 1989. Unification in Boolean Rings and Abelian Groups. *Journal of Symbolic Computation* 8, 5 (nov 1989), 449–477. https://doi.org/10.1016/s0747-7171(89)80054-9

Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. 2018. Effect handlers for the masses. *Proceedings of the ACM on Programming Languages* 2, OOPSLA (oct 2018), 1–27. https://doi.org/10.1145/3276481

Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. 2020. Effekt: Capability-passing style for type- and effect-safe, extensible effect handlers in Scala. *Journal of Functional Programming* 30 (2020). https://doi.org/10.1017/s0956796820000027

Edwin Brady. 2013. Programming and reasoning with algebraic effects and dependent types. In *Proceedings of the 18th ACM SIGPLAN international conference on Functional programming*. ACM. https://doi.org/10.1145/2500365.2500581

Randal E. Bryant. 1992. Symbolic Boolean Manipulation with Ordered Binary-Decision Diagrams. *Comput. Surveys* 24, 3 (1992), 293–318. https://doi.org/10.1145/136035.136043

Wolfram Buttner and Helmut Simonis. 1987. Embedding boolean expressions into logic programming. *Journal of Symbolic Computation* 4, 2 (oct 1987), 191–205. https://doi.org/10.1016/s0747-7171(87)80065-2

Lukas Convent, Sam Lindley, Conor McBride, and Craig McLaughlin. 2020. Doo bee doo bee doo. *Journal of Functional Programming* 30 (2020). https://doi.org/10.1017/s0956796820000039

Luis Damas. 1984. *Type assignment in programming languages*. Ph. D. Dissertation. The University of Edinburgh.

Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*. Springer Berlin Heidelberg, 337–340. https://doi.org/10.1007/978-3-540-78800-3_24

Edsko de Vries, Rinus Plasmeijer, and David M. Abrahamson. 2008. Uniqueness Typing Simplified. In *Implementation and Application of Functional Languages*. Springer Berlin Heidelberg, 201–218. https://doi.org/10.1007/978-3-540-85373-2_12

Benedict R Gaster and Mark P Jones. 1996. *A polymorphic type system for extensible records and variants*. Technical Report. Technical Report NOTTCS-TR-96-3, Department of Computer Science, University of Nottingham.

J.a. Gerhard and Mario Petrich. 1994. Unification in free distributive lattices. *Theoretical Computer Science* 126, 2 (apr 1994), 237–257. https://doi.org/10.1016/0304-3975(94)90011-6

Silvio Ghilardi. 1999. Unification in intuitionistic logic. *Journal of Symbolic Logic* 64, 2 (jun 1999), 859–880. https://doi.org/10.2307/2586506

Daniel Hillerström and Sam Lindley. 2016. Liberating effects with rows and handlers. In *Proceedings of the 1st International Workshop on Type-Driven Development*. ACM. https://doi.org/10.1145/2976022.2976033

Roger Hindley. 1969. The principal type-scheme of an object in combinatory logic. *Transactions of the American Mathematical Society (AMS)* (1969).

Mark P. Jones. 1994. A theory of qualified types. *Science of Computer Programming* 22, 3 (jun 1994), 231–256. https://doi.org/10.1016/0167-6423(94)00005-0

Ohad Kammar and Matija Pretnar. 2017. No value restriction is needed for algebraic effects and handlers. *Journal of Functional Programming* 27 (2017), e7. https://doi.org/10.1017/S0956796816000320

Oleg Kiselyov and Kc Sivaramakrishnan. 2018. Eff Directly in OCaml. *Electronic Proceedings in Theoretical Computer Science* 285 (dec 2018), 23–58. https://doi.org/10.4204/eptcs.285.2

Daan Leijen. 2005. Extensible records with scoped labels. *Trends in Functional Programming* 6 (2005), 179–194.

Daan Leijen. 2014. Koka: Programming with row polymorphic effect types. *Workshop on Mathematically Structured Functional Programming (MSFP)* (2014).

Daan Leijen. 2017. Type directed compilation of row-typed algebraic effects. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. ACM. https://doi.org/10.1145/3009837.3009872

Sam Lindley and James Cheney. 2012. Row-based effect types for database integration. In *Proceedings of the 8th ACM SIGPLAN workshop on Types in language design and implementation*. ACM. https://doi.org/10.1145/2103786.2103798

Sam Lindley, Conor McBride, and Craig McLaughlin. 2017. Do be do be do. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*. ACM. https://doi.org/10.1145/3009837.3009897

Alberto Lovato, Damiano Macedonio, and Fausto Spoto. 2014. A Thread-Safe Library for Binary Decision Diagrams. In *Software Engineering and Formal Methods*. Springer, 35–49. https://doi.org/10.1007/978-3-319-10431-7_4

Leopold Löwenheim. 1908. *Über das Auflösungsproblem im logischen Klassenkalkul.*

J. M. Lucassen and D. K. Gifford. 1988. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '88*. ACM Press. https://doi.org/10.1145/73560.73564

E. Macii, G. Odasso, and M. Poncino. 1998. Comparing different Boolean unification algorithms. In *Conference Record of Thirty-Second Asilomar Conference on Signals, Systems and Computers*. IEEE. https://doi.org/10.1109/acssc.1998.751423

Magnus Madsen and Ondřej Lhoták. 2020. Fixpoints for the masses: programming with first-class Datalog constraints. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (nov 2020), 1–28. https://doi.org/10.1145/3428193

Magnus Madsen and Jaco van de Pol. 2020. Polymorphic types and effects with Boolean unification. *Proceedings of the ACM on Programming Languages* 4, OOPSLA (nov 2020), 1–29. https://doi.org/10.1145/3428222

Magnus Madsen and Jaco van de Pol. 2021. Relational nullable types with Boolean unification. *Proceedings of the ACM on Programming Languages* 5, OOPSLA (oct 2021), 1–28. https://doi.org/10.1145/3485487

Magnus Madsen and Jaco van de Pol. 2023. Programming with Purity Reflection: Peaceful Coexistence of Effects, Laziness, and Parallelism. Schloss Dagstuhl - Leibniz-Zentrum für Informatik. https://doi.org/10.4230/LIPICS.ECOOP.2023.18

Magnus Madsen, Jaco van de Pol, and Troels Henriksen. 2023. *Fast and Efficient Boolean Unification for Hindley-Milner-Style Type and Effect Systems (artifact).* https://doi.org/10.5281/zenodo.8318658

Alberto Martelli and Ugo Montanari. 1982. An Efficient Unification Algorithm. *ACM Transactions on Programming Languages and Systems* 4, 2 (apr 1982), 258–282. https://doi.org/10.1145/357162.357169

Urusula Martin and Tobias Nipkow. 1989. Boolean unification — The story so far. *Journal of Symbolic Computation* 7, 3-4 (mar 1989), 275–293. https://doi.org/10.1016/s0747-7171(89)80013-6

E. J. McCluskey. 1956. Minimization of Boolean Functions. *Bell System Technical Journal* 35, 6 (nov 1956), 1417–1444. https://doi.org/10.1002/j.1538-7305.1956.tb03835.x

Robin Milner. 1978. A theory of type polymorphism in programming. *J. Comput. System Sci.* 17, 3 (dec 1978), 348–375. https://doi.org/10.1016/0022-0000(78)90014-4

Leonardo De Moura and Nikolaj Bjørner. 2011. Satisfiability modulo theories. *Commun. ACM* 54, 9 (sep 2011), 69–77. https://doi.org/10.1145/1995376.1995394

Flemming Nielson, Hanne Riis Nielson, and Chris Hankin. 1999. Type and Effect Systems. In *Principles of Program Analysis*. Springer Berlin Heidelberg, 283–363. https://doi.org/10.1007/978-3-662-03811-6_5

Martin Odersky, Martin Sulzmann, and Martin Wehr. 1999. Type inference with constrained types. *Theory and Practice of Object Systems* 5, 1 (jan 1999), 35–55. https://doi.org/10.1002/(sici)1096-9942(199901/03)5:1<35::aid-tapo4>3.0.co;2-4

Gordon Plotkin and Matija Pretnar. 2009. Handlers of Algebraic Effects. In *Programming Languages and Systems*. Springer Berlin Heidelberg, 80–94. https://doi.org/10.1007/978-3-642-00590-9_7

Matija Pretnar. 2014. Inferring Algebraic Effects. *Logical Methods in Computer Science* Volume 10, Issue 3 (sep 2014). https://doi.org/10.2168/lmcs-10(3:21)2014

Matija Pretnar. 2015. An Introduction to Algebraic Effects and Handlers. Invited tutorial paper. *Electronic Notes in Theoretical Computer Science* 319 (dec 2015), 19–35. https://doi.org/10.1016/j.entcs.2015.12.003

W. V. Quine. 1952. The Problem of Simplifying Truth Functions. *The American Mathematical Monthly* 59, 8 (oct 1952), 521–531. https://doi.org/10.1080/00029890.1952.11988183

Alan Ja Robinson and Andrei Voronkov. 2001. *Handbook of automated reasoning*. Vol. 1. Elsevier.

Sergiu Rudeanu. 1974. *Boolean functions and equations.*

Lukas Rytz, Martin Odersky, and Philipp Haller. 2012. Lightweight Polymorphic Effects. In *ECOOP 2012 – Object-Oriented Programming*. Springer Berlin Heidelberg, 258–282. https://doi.org/10.1007/978-3-642-31057-7_13

Helmut Simonis and Mehmet Dincbas. 1987. Using an extended Prolog for digital circuit design. In *IEEE International Workshop on AI Applications to CAD Systems for Electronics.*

Martin Sulzmann. 2001. A General Type Inference Framework for Hindley/Milner Style Systems. In *Functional and Logic Programming*. Springer Berlin Heidelberg, 248–263. https://doi.org/10.1007/3-540-44716-4_16

Mads Tofte and Jean-Pierre Talpin. 1997. Region-Based Memory Management. *Information and Computation* 132, 2 (feb 1997), 109–176. https://doi.org/10.1006/inco.1996.2613

P. Wadler and S. Blott. 1989. How to make ad-hoc polymorphism less ad hoc. In *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages - POPL '89*. ACM Press. https://doi.org/10.1145/75277.75283

Keith Wansbrough and Simon Peyton Jones. 1999. Once upon a polymorphic type. In *Proceedings of the 26th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. ACM. https://doi.org/10.1145/292540.292545

John Whaley. 2023. *JavaBDD.* https://javabdd.sourceforge.net/

Received 2023-04-14; accepted 2023-08-27